

```
!pip install -U spacy
```

```
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.11)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (8.3.10)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.5.2)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.4.3)
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.20.0)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (4.67.1)
Requirement already satisfied: numpy==1.19.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.2)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4 in /usr/local/lib/python3.12/dist-packages (from spacy)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from spacy) (75.2.0)
Requirement already satisfied: packaging>20.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (25.0)
Requirement already satisfied: annotated-types==0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1)
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy)
Requirement already satisfied: certifi=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy)
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4->spacy)
Requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4)
Requirement already satisfied: click==8.0.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim<1.0.0,>=0.3.0->spacy)
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.2)
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.2)
Requirement already satisfied: MarkupSafe==2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->spacy) (3.0.3)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open<8.0.0,>=5.2.1->weasel<0.5.0,>=0.4.2)
```

```
!python -m spacy download en_core_web_md
```

```
Collecting en-core-web-md==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en\_core\_web\_md-3.8.0/en\_core\_web\_md-3.8.0-py3-none-any.whl
    33.5/33.5 MB 44.2 MB/s eta 0:00:00

✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_md')

⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in order to load all the package's dependencies. You can do this by selecting the 'Restart kernel' or 'Restart runtime' option.
```

2.1. Tải mô hình và phân tích câu

```
import spacy
from spacy import displacy
import en_core_web_md

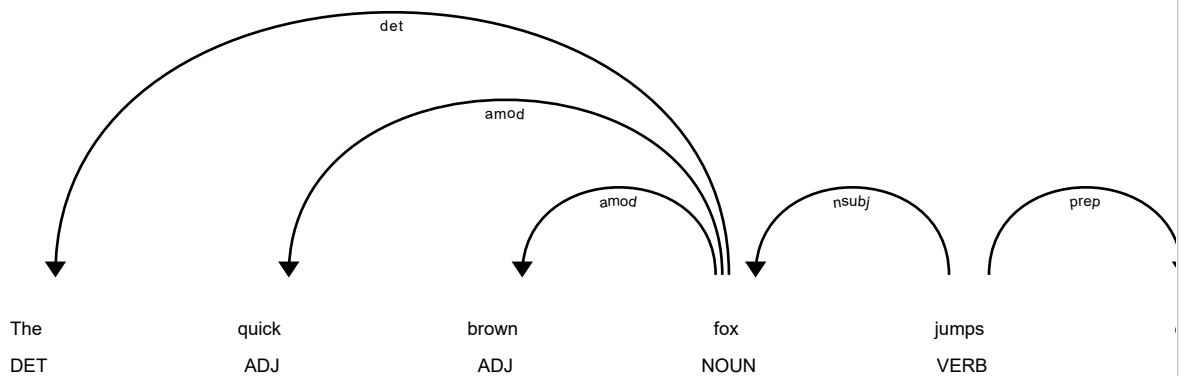
# Tải mô hình tiếng Anh đã cài đặt
# Sử dụng en_core_web_md vì nó chứa các vector từ và cây cú pháp đầy đủ
nlp = en_core_web_md.load()

# Câu ví dụ
text = "The quick brown fox jumps over the lazy dog."

# Phân tích câu với pipeline của spaCy
doc = nlp(text)
```

```
displacy.serve(doc, style="dep")
```

```
/usr/local/lib/python3.12/dist-packages/spacy/displacy/__init__.py:108: UserWarning: [W011] It looks like you're calling dis
warnings.warn(Warnings.W011)
```



Using the 'dep' visualizer
 Serving on <http://0.0.0.0:5000> ...
 Shutting down server on port 5000.

- Từ nào là gốc (ROOT) của câu : jumps
- jumps có những từ phụ thuộc (dependent) và quan hệ: fox(usubj) , over(pre)
- fox là head của những từ: The, quick, brown

▼ Phần 3: Truy cập các thành phần trong cây phụ thuộc

```
# Lấy một câu khác để phân tích
text = "Apple is looking at buying U.K. startup for $1 billion"
doc = nlp(text)
# In ra thông tin của từng token
print(f"{'TEXT':<12} | {'DEP':<10} | {'HEAD TEXT':<12} | {'HEAD POS':<8} | {'CHILDREN'}")
print("-" * 70)

for token in doc:
    # Trích xuất các thuộc tính
    children = [child.text for child in token.children]
    print(f"{token.text:<12} | {token.dep_:<10} | {token.head.text:<12} | {token.head.pos_:<8}")
```

TEXT	DEP	HEAD TEXT	HEAD POS	CHILDREN
Apple	nsubj	looking	VERB	
is	aux	looking	VERB	
looking	ROOT	looking	VERB	
at	prep	looking	VERB	
buying	pcomp	at	ADP	
U.K.	compound	startup	NOUN	
startup	dobj	buying	VERB	
for	prep	startup	NOUN	
\$	quantmod	billion	NUM	
1	compound	billion	NUM	
billion	pobj	for	ADP	

▼ Phần 4: Duyệt cây phụ thuộc để trích xuất thông tin

4.1. Bài toán: Tìm chủ ngữ và tân ngữ của một động từ

```
text = "The cat chased the mouse and the dog watched them."
doc = nlp(text)
```

```

for token in doc:
# Chỉ tìm các động từ
    if token.pos_ == "VERB":
        verb = token.text
        subject = ""

        obj = ""
        # Tìm chủ ngữ (nsubj) và tân ngữ (dobj) trong các con của động từ
        for child in token.children:
            if child.dep_ == "nsubj":
                subject = child.text
            if child.dep_ == "dobj":
                obj = child.text
        if subject and obj:
            print(f"Found Triplet: ({subject}, {verb}, {obj})")

```

```

Found Triplet: (cat, chased, mouse)
Found Triplet: (dog, watched, them)

```

4.2. Bài toán: Tìm các tính từ bổ nghĩa cho một danh từ

```

text = "The big, fluffy white cat is sleeping on the warm mat."
doc = nlp(text)
for token in doc:
# Chỉ tìm các danh từ
    if token.pos_ == "NOUN":
        adjectives = []
        # Tìm các tính từ bổ nghĩa (amod) trong các con của danh từ
        for child in token.children:
            if child.dep_ == "amod":
                adjectives.append(child.text)
        if adjectives:
            print(f"Danh từ '{token.text}' được bổ nghĩa bởi các tính từ: {adjectives}")

```

```

Danh từ 'cat' được bổ nghĩa bởi các tính từ: ['big', 'fluffy', 'white']
Danh từ 'mat' được bổ nghĩa bởi các tính từ: ['warm']

```

Phần 5: Bài tập tự luyện

Bài 1: Tìm động từ chính của câu

```

def find_main_verb(doc):
    for token in doc:
        if token.dep_ == "ROOT":
            return token
    return None

doc_ex1 = nlp("The quick brown fox jumps over the lazy dog.")
main_verb = find_main_verb(doc_ex1)
print(f"Động từ chính: {main_verb.text} (POS: {main_verb.pos_})")

```

```

Động từ chính: jumps (POS: VERB)

```

Bài 2: Trích xuất các cụm danh từ (Noun Chunks)

```

def get_subtree_text(token):
    """Hàm đệ quy để lấy toàn bộ từ ngữ trong nhánh của token theo đúng thứ tự"""
    # Lấy token hiện tại và tất cả con cháu của nó (subtree)
    # Sắp xếp lại theo chỉ số (index) xuất hiện trong câu để cụm từ có nghĩa
    words = sorted([t for t in token.subtree], key=lambda t: t.i)
    return " ".join([t.text for t in words])

def custom_noun_chunks(doc):
    chunks = []
    for token in doc:
        # Tìm các danh từ (NOUN) hoặc danh từ riêng (PROPN)
        # Và đảm bảo nó không phải là con của một danh từ khác (để tránh lấy cụm chồng cụm)
        if token.pos_ in ["NOUN", "PROPN"]:
            # Logic đơn giản: Lấy toàn bộ nhánh con của danh từ này
            # Lưu ý: Đây là cách đơn giản, thực tế spaCy có logic phức tạp hơn
            chunk_text = get_subtree_text(token)
            chunks.append(chunk_text)
    return chunks

```

```
# Test
doc_ex2 = nlp("The quick brown fox jumps over the lazy dog")
print(f"Bài 2 - Các cụm danh từ (tự viết): {custom_noun_chunks(doc_ex2)}")
# So sánh với spaCy có sẵn
print(f"Bài 2 - Các cụm danh từ (spaCy): {[chunk.text for chunk in doc_ex2.noun_chunks]}")
```

```
Bài 2 - Các cụm danh từ (tự viết): ['The quick brown fox', 'the lazy dog']
Bài 2 - Các cụm danh từ (spaCy): ['The quick brown fox', 'the lazy dog']
```

▼ Bài 3: Tìm đường đi ngắn nhất trong cây

```
def get_path_to_root(token):
    """Trả về danh sách token từ token hiện tại lên đến ROOT."""
    path = [token]
    current = token
    # Duyệt ngược lên head cho đến khi gặp chính nó (Root head là chính nó)
    # hoặc check current.dep_ == 'ROOT'
    while current.head != current:
        current = current.head
        path.append(current)
    return path
```

```
# Test
doc_ex3 = nlp("The quick brown fox jumps over the lazy dog")
# Giả sử ta muốn tìm đường đi từ từ "startup" lên gốc
token_startup = doc_ex3[2] # vị trí của từ startup
path = get_path_to_root(token_startup)
```

```
print("Bài 3 - Đường đi từ 'startup' đến ROOT:")
# In ra đường đi với mũi tên
path_text = " -> ".join([t.text for t in path])
print(path_text)
```

```
Bài 3 - Đường đi từ 'startup' đến ROOT:
brown -> fox -> jumps
```